

Estimación de la temperatura de punto de rocío (TDEW) a partir de la temperatura mínima sobre PISCOT

Motivación, problema y metodología: regresión lineal local sobre anomalías con un diseño paralelo bucket-año

Piero Palacios Bernuy

Estuardo Oliver Campos

Diego Sánchez Salazar

Gianmarco Mejía

2026-06-13

Resumen La temperatura de punto de rocío (TDEW, en adelante TD) es una variable climática fundamental: determina la presión de vapor real del aire y, con ello, la formación de rocío, el riesgo de heladas y la duración del período de humectación foliar que controla la infección por patógenos fúngicos en los cultivos. Como la TD rara vez se mide de forma directa en redes meteorológicas extensas, debe estimarse a partir de variables más disponibles, típicamente la temperatura mínima diaria (TMIN). Este trabajo aborda dicha estimación sobre el conjunto de datos grillados PISCOT para el Perú, comparando sus versiones v1.1 y v1.2, lo que implica ajustar un modelo para del orden de 300 000 ubicaciones espaciales a lo largo de cerca de 40 años de registro diario. Proponemos un modelo de regresión lineal local ponderada sobre anomalías, analizado en el modelo PRAM (variante CREW) e implementado con un rediseño *bucket-año* que reduce drásticamente el sobrecosto de entrada/salida (E/S). Finalmente, se define un protocolo experimental de escalamiento fuerte y débil que mide tiempo de ejecución, aceleración y eficiencia en CPU y GPU (NVIDIA A100), ejecutado de forma idéntica sobre PISCOT v1.1 y v1.2.

Palabras clave: temperatura de punto de rocío, temperatura mínima, regresión lineal local, anomalías climáticas, PISCOT, PRAM, escalamiento HPC, enfermedades fúngicas.

Tabla de contenidos

1	Introducción	2
1.1	La importancia de la temperatura de punto de rocío	2
1.2	El conjunto de datos PISCOT	3
1.3	El problema a resolver	3
1.4	Contribución y estructura	3
2	Metodología	4
2.1	Datos y variables	4
2.2	Modelo de regresión lineal local	4
2.3	Modelo PRAM y notación de complejidad	5
2.4	Pipeline completo	5

2.5	Algoritmo dominante: entrenamiento de coeficientes	6
2.6	Complejidad global	7
2.7	Implementación y ejecución de los experimentos	7
2.7.1	Qué cuenta como un procesador p	7
2.7.2	Cantidades medidas	8
2.7.3	Escalamiento fuerte y débil	8
2.7.4	Controles y reportes	8
	Referencias	8
	Apéndice A — Pseudocódigo PRAM y análisis de complejidad	10
2.8	A.2 Modelo PRAM	10
2.9	A.3 Símbolos	10
2.10	A.4 Cómo leer las fórmulas de complejidad	10
2.11	A.6 Algoritmo 2: Construir Climatología	10
2.12	A.7 Algoritmo 3: Construir el Dataset de Entrenamiento Bucket-Año	11
2.13	A.8 Algoritmo 4: Fragmentar Climatología por Bucket	11
2.14	A.9 Algoritmo 4b: Fragmentar TMIN Futuro por Bucket	12
2.15	A.10 Algoritmo 5: Entrenar Coeficientes de Anomalía	12
2.16	A.11 Algoritmo 6: Verificar Completitud	13
2.17	A.12 Algoritmo 7: Reparar DOYs Fallidos	13
2.18	A.13 Algoritmo 8: Parchar Coeficientes	14
2.19	A.14 Algoritmo 9: Pronóstico Opcional (bucketizado)	14
2.20	A.15 Diseño antiguo vs diseño bucket-año	15
2.21	A.17 Complejidad global	15
2.22	A.18 Interpretación HPC	15
2.23	A.19 El hardware es simplemente más procesadores	16
2.24	A.20 Metodología experimental	16

1 Introducción

1.1 La importancia de la temperatura de punto de rocío

La temperatura de punto de rocío es la temperatura a la cual el aire debe enfriarse, a presión constante, para que el vapor de agua que contiene alcance la saturación y comience a condensarse. Es, por tanto, una medida directa del contenido de humedad del aire y fija la presión de vapor real empleada en el cálculo del balance hídrico y de la evapotranspiración de referencia mediante la ecuación de Penman–Monteith (Allen et al., 1998). Su relevancia abarca desde la hidrología y la predicción de heladas hasta, de manera especialmente crítica, la fitopatología agrícola.

El vínculo con la sanidad vegetal es directo. La mayoría de los hongos y bacterias fitopatógenos requieren agua líquida libre sobre la superficie de la planta para germinar e infectar; esa agua proviene de la lluvia, la niebla y, sobre todo, del rocío, que se forma cuando la temperatura de la superficie foliar desciende por debajo del punto de rocío del aire circundante (Huber & Gillespie, 1992). La **duración del período de humectación foliar** —principal predictor del riesgo de infección— suele estimarse a partir de variables meteorológicas como la depresión del punto de rocío y la humedad relativa, precisamente porque medirla de forma directa es costoso (Sentelhas et al., 2008). En consecuencia, disponer de campos espacialmente continuos y fiables de TD es un insumo de primer orden para los sistemas de alerta temprana de enfermedades.

Este punto es particularmente importante en el contexto peruano y andino. La papa, cultivo estratégico para la seguridad alimentaria de la región, es severamente afectada por el tizón tardío (*Phytophthora infestans*), cuya tasa de infección y esporulación depende de manera estrecha de la temperatura y de

la humedad: las noches frescas con rocío abundante seguidas de días húmedos constituyen las condiciones ideales para la epidemia (Narouei-Khandan et al., 2020). Los modelos mecanicistas de pronóstico de tizón se alimentan explícitamente de series de temperatura y humedad, de modo que la calidad de las estimaciones de TD condiciona directamente la utilidad de estas herramientas de gestión.

1.2 El conjunto de datos PISCOT

PISCOT es el producto grillado de temperatura del aire para el Perú desarrollado por el Servicio Nacional de Meteorología e Hidrología (SENAMHI) dentro de la familia PISCO (*Peruvian Interpolated data of SENAMHI's Climatological and hydrological Observations*). Su versión más reciente, PISCOT v1.2, ofrece grillas diarias de temperatura del aire (máxima y mínima) a una resolución espacial de 0.01° para todo el territorio nacional en el período 1981–2020 (Huerta et al., 2023). La construcción del producto sigue cuatro etapas: control de calidad, relleno de vacíos, homogeneización de las estaciones y, finalmente, interpolación espacial; la versión v1.2 mejora sobre la v1.1 mediante datos adicionales, una secuencia de cálculo revisada y un control de versiones más estricto (Huerta et al., 2023).

Para este proyecto, PISCOT aporta la variable predictora (TMIN) sobre una grilla densa y homogénea. La temperatura de punto de rocío, en cambio, tiene una disponibilidad limitada e inconsistente entre versiones: **PISCOT v1.1 incluye TD solo hasta 2016, mientras que PISCOT v1.2 no incluye TD en absoluto**. Esta cobertura parcial es precisamente lo que motiva estimar la TD a partir de la TMIN, de modo que se obtenga un campo de TD completo y homogéneo para todo el período y coherente entre ambas versiones del producto. Comparar v1.1 y v1.2 permite además evaluar cómo los cambios en el producto subyacente se propagan al modelo de TD estimado.

1.3 El problema a resolver

El objetivo es estimar la TD diaria para cada celda de la grilla PISCOT usando la TMIN como predictor principal. La escala es masiva: del orden de $N \approx 300\,000$ ubicaciones espaciales (identificadas por un ID) a lo largo de $Y \approx 40$ años de registro diario, lo que supone del orden de miles de millones de registros por variable climática.

El reto no es solo de volumen, sino también estadístico. La temperatura cruda —tanto TD como TMIN— posee un ciclo estacional fuerte y predecible: cada ubicación es más cálida en verano y más fría en invierno. Si se modelara la TD cruda frente a la TMIN cruda, gran parte de la relación aparente sería simplemente el reflejo de que ambas variables siguen las estaciones, y no el acoplamiento físico genuino entre humedad y temperatura que interesa capturar. Por ello se trabaja sobre **anomalías**, restando a cada observación el valor típico de su ubicación para ese día del año (su climatología), tal como se define en la Ecuación 1:

$$\begin{aligned} TD_{anom}(t) &= TD(t) - TD_{clim}(ID, doy), \\ TMIN_{anom}(t) &= TMIN(t) - TMIN_{clim}(ID, doy). \end{aligned} \tag{1}$$

Modelar la relación día a día por separado para cada par (ID, doy) genera del orden de $N \times D \approx 300\,000 \times 366 \approx 1.1 \times 10^8$ ajustes de regresión independientes. Cada ajuste es diminuto, pero su número total convierte el cómputo en un problema intensivo que exige un diseño paralelo cuidadoso y motiva tanto el análisis de complejidad como los experimentos de escalamiento descritos en la metodología.

1.4 Contribución y estructura

El trabajo propone (i) un modelo de regresión lineal local ponderada sobre anomalías para estimar TD a partir de TMIN; (ii) un rediseño **bucket-año** del flujo de datos que elimina el reescaneo repetido de archivos fuente del enfoque anterior; (iii) un análisis de complejidad en el modelo PRAM que predice el rendimiento en clústeres de cómputo de alto desempeño (HPC); y (iv) un protocolo experimental que

mide el escalamiento fuerte y débil en CPU y GPU sobre las dos versiones de PISCOT. La Sección de Metodología detalla los datos, el modelo estadístico, el modelo PRAM con su pseudocódigo y análisis de complejidad, la implementación y el diseño experimental.

2 Metodología

2.1 Datos y variables

La unidad de análisis es la celda de grilla (**ID**). De PISCOT (Huerta et al., 2023) se toman las series diarias de TMIN (predictor) y TD (objetivo) para cada **ID** durante el período de entrenamiento. Todo el protocolo se ejecuta de forma idéntica sobre PISCOT v1.1 y v1.2, suministrando cada versión al mismo pipeline mediante parámetros de ruta de entrada y escribiendo a raíces de resultados separadas, de modo que no difiere ningún código entre ejecuciones.

Para cada **ID** y día del año **doy** se define la climatología como la media histórica de cada variable, y las anomalías como la desviación respecto de dicha climatología. La regresión se ajusta sobre las anomalías, eliminando así el ciclo estacional. La notación empleada en todo el documento se resume en la Tabla 1.

Tabla 1: Símbolos y parámetros del modelo.

Símbolo	Significado	Valor típico
N	Número de ubicaciones espaciales (ID)	$\approx 300\,000$
Y	Años de entrenamiento	≈ 40
D	Días por año	≤ 366
B	Número de <i>buckets</i> de ID	p. ej. 1024
p	Procesadores / trabajadores	variable
h	Semiventana local de doy	11
F	Coefficientes de regresión (intercepto + predictores)	5
m	Muestras por regresión local	$O(Yh)$

2.2 Modelo de regresión lineal local

El núcleo estadístico es una **regresión lineal local ponderada**: un método de regresión no paramétrica que, en lugar de ajustar una sola recta global, ajusta un modelo lineal simple dentro de una vecindad local de cada punto, ponderando las observaciones según su cercanía mediante una función de kernel (Cleveland, 1979; Fan & Gijbels, 1996). Para estimar el modelo de la ubicación **i** en el día del año **d**, no se usan únicamente las observaciones del día **d** —demasiado escasas a lo largo de 40 años—, sino todas aquellas cuyo día del año cae dentro de una **ventana circular local** de semianchura **h** alrededor de **d**, de modo que el día 365 es vecino del día 1.

Las observaciones más próximas a **d** reciben mayor influencia mediante un peso **tricube** que decae suavemente hasta cero en el borde de la ventana (Ecuación 2):

$$w(t) = \left(1 - \left|\frac{\delta(\text{doy}(t), d)}{h}\right|^3\right)^3, \quad \text{para } |\delta| \leq h, \quad (2)$$

donde δ es la distancia circular entre días del año. Sobre esa vecindad ponderada se resuelve una regresión por mínimos cuadrados ponderados de la anomalía de TD sobre la anomalía de TMIN y unas pocas características de rezago (**TD_anom_lag1**, **TD_anom_lag2**, **TMIN_anom_lag1**), obteniendo el vector

de coeficientes β que se almacena para (i, d). Con $F = 5$ coeficientes (intercepto más 4 predictores) y $m = O(Yh)$ muestras de vecindad por ajuste, cada ajuste resuelve el sistema de ecuaciones normales

$$(X^T W X) \beta = X^T W y. \quad (3)$$

2.3 Modelo PRAM y notación de complejidad

El análisis paralelo se realiza en el modelo **PRAM** (Máquina de Acceso Aleatorio Paralela), el marco idealizado estándar para razonar sobre algoritmos paralelos (Cormen et al., 2009; JáJá, 1992). Se usa la variante **CREW** (*Concurrent Read, Exclusive Write*), que se ajusta de forma natural a este pipeline: muchos procesadores pueden leer concurrentemente las mismas tablas de entrada (lectura concurrente), mientras que cada trabajador es dueño de un bucket / ID / día del año de salida disjunto, de modo que nunca colisionan dos escrituras (escritura exclusiva). Las reducciones (sumas, medias, conteos) combinan m valores en aproximadamente $\log m$ pasos mediante un árbol de sumas por pares. La anotación **pardo** marca los bucles cuyas iteraciones son independientes y, por tanto, paralelizables.

Cada algoritmo se describe con tres costos (JáJá, 1992): el **trabajo** W (número total de operaciones, independiente del número de procesadores), el **tiempo paralelo** T_p con p procesadores, y el **span** T_∞ (camino crítico, el tiempo con procesadores ilimitados). La forma recurrente del tiempo paralelo es

$$T_p \approx \frac{W}{p_{\text{eff}}} + \text{sobrecosto (coordinación, E/S, dependencias)}.$$

El **diseño bucket-año** es la decisión arquitectónica central. Un *bucket* es un grupo determinista de IDs definido por $\text{bucket} = \text{ID} \bmod B$. En lugar de que cada uno de los ~300 000 IDs vuelva a abrir los mismos archivos mensuales históricos, los datos se agrupan una sola vez en B buckets y se compactan por año; después, cada trabajador lee los archivos compactos de su bucket una única vez, convirtiendo una tormenta de lecturas diminutas y repetidas en unas pocas lecturas secuenciales grandes. La principal consecuencia es que el paralelismo efectivo de las etapas de cómputo queda acotado por el número de buckets, $p_{\text{eff}} \leq B$.

2.4 Pipeline completo

El pipeline encadena nueve algoritmos; cada uno se ejecuta en secuencia, pero es internamente paralelo. El Algoritmo 1 da la vista general.

Algorithm TDEW-PIPELINE

Input:

TD, TMIN monthly files for all IDs and dates; Grid of expected IDs

Parameters: B buckets, h local DOY half-window, p processors

Output:

climatology; bucket-year training dataset;

regression coefficients per (ID, doy); optional repairs; optional TD predictions

```

1  climatology      <- BUILD-CLIMATOLOGY(TD, TMIN)
2  bucketed_training <- BUILD-BUCKET-YEAR-DATASET(TD, TMIN, B)
3  bucketed_clim    <- SHARD-CLIMATOLOGY(climatology, B)
4  coefficients      <- TRAIN-COEFFICIENTS(bucketed_training, bucketed_clim, h, B)
5  incomplete_doy    <- CHECK-COMPLETENESS(coefficients, Grid)
6  if incomplete_doy is not empty then
7      patch          <- RERUN-FAILED-DOYS(bucketed_training, bucketed_clim,
incomplete_doy, h, B)
8      coefficients    <- PATCH-COEFFICIENTS(coefficients, patch, incomplete_doy)
```

```

9  end if
10 if forecasting requested then
11     bucketed_future_tmin <- SHARD-FUTURE-TMIN(TMIN_future, B)
12     predictions <- FORECAST-TD(coefficients, bucketed_clim, bucketed_training,
13                               bucketed_future_tmin, H, B)
14 end if
15 return coefficients, predictions

```

El flujo aprende primero los valores normales estacionales de cada ubicación (climatología, línea 1); reorganiza los archivos crudos en archivos compactos por bucket (líneas 2–3); ajusta las regresiones locales que constituyen la salida científica (línea 4); ejecuta un control de calidad que reentrena solo los días faltantes (líneas 5–9); y, opcionalmente, proyecta el modelo hacia adelante sobre un horizonte futuro (líneas 10–13).

2.5 Algoritmo dominante: entrenamiento de coeficientes

La etapa que domina el costo es el ajuste de las regresiones locales. Para cada ID y cada día del año objetivo, se reúne la vecindad circular de días, se eliminan filas incompletas, se ponderan con el kernel tricube y se resuelve la regresión.

```

Algorithm TRAIN-COEFFICIENTS
Input: bucket-year training dataset; bucketed climatology; half-window h; buckets B
Output: coefficients per (ID, doy)

1  for each bucket b in 0..B-1 pardo do
2      train_b <- read bucket-year training rows for bucket b
3      clim_b <- read climatology rows for bucket b
4      for each ID i in bucket b pardo do
5          compute TD_anom, TMIN_anom for i (subtract climatology)
6          create lag features (TD_anom_lag1, TD_anom_lag2, TMIN_anom_lag1)
7          for each target day d in 1..366 pardo do
8              N_d <- rows with circular_distance(doy, d) <= h, complete cases
9              if size(N_d) >= minimum_sample_count then
10                 weight(t) <- tricube(circular_distance(doy(t), d) / h)
11                 beta <- weighted_least_squares(N_d)
12                 write coefficients for (ID i, doy d)
13             end if
14         end for
15     end for
16     write coefficient file for bucket b
17 end for
18 return coefficient dataset

```

Como $F = 5$ es constante, un ajuste cuesta $W_{\text{one-fit}} = O(mF^2 + F^3) = O(m) = O(Yh)$. El trabajo total del entrenamiento es, pues, el costo de un ajuste multiplicado por el número de ajustes (N ubicaciones $\times D$ días del año), según la Ecuación 4:

$$W_{\text{train}} = O(N D Y h). \quad (4)$$

Con procesadores ilimitados el span es $T_{\infty} = O(\log m + F^3)$ —el algoritmo es extremadamente “ancho”—, mientras que con p procesadores y B tareas de bucket el tiempo práctico es

$$T_p = \left(O \frac{NDYh}{\min(p, B)} + \text{sobrecosto de E/S} \right).$$

2.6 Complejidad global

Sumando las fases dominantes (preprocesamiento bucket-año y entrenamiento completo), con $R = NYD$, el trabajo total es

$$W_{\text{total}} = \left(OR \log \frac{ND}{B} + NDYh \right) = O(NDYh),$$

ya que el factor h de la regresión local es el multiplicador dominante. El tiempo paralelo práctico combina las fases limitadas por entrada/salida (E/S, es decir, lectura y escritura de archivos en disco; que escalan con $/p$) y las etapas de cómputo bucket-paralelas (que escalan con $/\min(p, B)$). La Tabla 2 recopila los costos por fase.

Tabla 2: Trabajo y tiempo paralelo por fase del pipeline.

Fase	Trabajo secuencial W	Tiempo paralelo T_p
Construir climatología	$O(R)$	$O(R/p + \log R)$
Dataset bucket-año	$O(R \log(ND/B))$	$O(R/p + \log(ND/B))$
Entrenar coeficientes	$O(NDYh)$	$O(NDYh / \min(p, B))$
Verificar completitud	$O(ND)$	$O(ND/p + \log(ND))$
Reparar Q DOYs	$O(NQYh)$	$O(NQYh / \min(p, B))$
Pronóstico (horizonte H)	$O(NH)$	$O(NH / \min(p, B) + H)$

El cuello de botella práctico tras el rediseño es $O(NDYh)$; el aporte del bucket-año es, sobre todo, eliminar la E/S repetida del enfoque anterior, no reducir el trabajo estadístico.

2.7 Implementación y ejecución de los experimentos

La implementación se apoya en **Dask** para la planificación de tareas y la paralelización, tanto en CPU (mediante `dask_jobqueue.SLURMCluster`) como en GPU (mediante `dask-cuda`) (Rocklin, 2015). Conviene distinguir dos niveles de paralelismo. El **nivel externo** (bucket) es el número de buckets que corren a la vez, acotado por $\min(p, B)$: en CPU, p es el número de procesos trabajadores de un hilo ($\approx$ núcleos); en GPU, p es el número de dispositivos A100, a menudo solo 1–8. El **nivel interno** (ajuste) es cuántos de los $\approx (N/B) D$ ajustes de un bucket se evalúan juntos: en CPU es uno a la vez (BLAS fijado a un hilo), mientras que una GPU ensambla y resuelve del orden de 10^5 sistemas 5×5 en una sola llamada por lotes. Por ello la CPU prefiere **muchos** buckets ($B \geq 4p$) para mantener ocupados los núcleos, mientras que la GPU prefiere buckets **lo bastante grandes** como para llenar cada lote.

2.7.1 Qué cuenta como un procesador p

El eje de escalamiento p cuenta **trabajadores independientes**, no hilos, para que $S(p)$ y $E(p)$ signifiquen lo mismo en ambos hardwares. En CPU, un trabajador es un proceso Dask de un solo hilo (un núcleo, con las bibliotecas BLAS fijadas a un hilo mediante `OMP_NUM_THREADS=1`), y p se varía escalando el número de procesos. En GPU, un trabajador es una GPU A100 completa (un proceso `dask-cuda` por dispositivo), de modo que p es el número de A100 de la asignación y se varía solicitando más dispositivos.

El paralelismo *interno* de la GPU —los miles de hilos CUDA y su organización en bloques— no se expone como p ni se barre como eje de escalamiento. La ruta de producción es un kernel CUDA propio (`solve5`:

una resolución de Cholesky 5×5 en registros, un hilo por ajuste) que recibe los $\approx 10^5$ sistemas 5×5 de un bucket y los reparte entre los multiprocesadores (SM) del dispositivo. El número de hilos por bloque es un parámetro de lanzamiento fijo (128 en este estudio), afinado una sola vez; cambiarlo no altera los resultados, solo el tiempo de reloj. Esa configuración intra-dispositivo se absorbe en la constante c del modelo $T_p = c(W/p_{\text{eff}}) + \text{sobrecosto}$, por lo que no es la variable de escalamiento. Lo único que controla la *carga* ofrecida a una sola GPU es el **ancho de lote** M : el número de ajustes resueltos por llamada al kernel, igual al número de ajustes por bucket, $M \approx (N/B) D$. Por ello, como estudio complementario al barrido de p , se barre el ancho de lote M (equivalentemente, se reduce B) para localizar el punto en que una sola A100 se satura.

2.7.2 Cantidades medidas

Para un tamaño de problema fijo, con $T(p)$ el tiempo de reloj usando p procesadores, se reportan tres cantidades estándar de rendimiento paralelo:

$$\text{Tiempo: } T(p), \quad \text{Aceleración: } S(p) = \frac{T(1)}{T(p)}, \quad \text{Eficiencia: } E(p) = \frac{S(p)}{p}.$$

La aceleración ideal es $S(p) = p$ y la eficiencia ideal $E(p) = 1$. El análisis PRAM predice la desviación: como las etapas de entrenamiento son bucket-paralelas con $p_{\text{eff}} \leq B$, la eficiencia se mantiene alta mientras $p \leq B$ y luego se degrada.

2.7.3 Escalamiento fuerte y débil

El **escalamiento fuerte** fija el tamaño del problema (la grilla completa o un subconjunto representativo fijo de IDs) y varía $p \in \{1, 2, 4, 8, 16, 32, \dots\}$ procesos en CPU y $p \in \{1, 2, 4, 8\}$ GPUs A100, registrando $T(p)$, $S(p)$ y $E(p)$ para la fase dominante de entrenamiento. Se espera una curva casi lineal mientras $p \leq B$ que luego se aplana, probando directamente la predicción $p_{\text{eff}} \leq B$.

El **escalamiento débil** mantiene constante el trabajo por procesador escalando el número de IDs entrenados con p , es decir $n(p) = n_0 \cdot p$, y reporta $E_{\text{weak}}(p) = T(1, n_0)/T(p, n_0 p)$, idealmente igual a 1. Como los ajustes por (ID, doy) son independientes, se espera buen escalamiento débil; las desviaciones aíslan el sobrecosto de E/S y de planificación que crece con el número de trabajadores.

2.7.4 Controles y reportes

Para garantizar mediciones reproducibles y justas: la línea base $T(1)$ usa un único trabajador con un hilo y los pools de BLAS fijados a 1; el número de buckets se mantiene en $B \geq 4 p_{\text{max}}$ durante todo el barrido; cada punto se ejecuta varias veces y se reporta la mediana; el almacenamiento temporal va a disco local rápido del nodo; y se distingue entre caché caliente y fría. El protocolo completo se ejecuta sobre **PISCOT v1.1 y v1.2**, comparando tanto el rendimiento (curvas $T(p)$, $S(p)$, $E(p)$) como la ciencia (distribuciones de los coeficientes ajustados y de la habilidad predictiva), para evaluar cómo la versión del dataset afecta al modelo estimado con independencia del tiempo de ejecución.

Referencias

- Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998). *Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements* (FAO Irrigation and Drainage Paper 56). FAO.
- Cleveland, W. S. (1979). Robust Locally Weighted Regression and Smoothing Scatterplots. *Journal of the American Statistical Association*, 74(368), 829–836.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Fan, J., & Gijbels, I. (1996). *Local Polynomial Modelling and Its Applications*. Chapman & Hall/CRC.
- Huber, L., & Gillespie, T. J. (1992). Modeling Leaf Wetness in Relation to Plant Disease Epidemiology. *Annual Review of Phytopathology*, 30, 553–577.

- Huerta, A., Aybar, C., Imfeld, N., Correa, K., Felipe-Obando, O., Rau, P., Drenkhan, F., & Lavado-Casimiro, W. (2023). High-Resolution Grids of Daily Air Temperature for Peru — the New PISCOt v1.2 Dataset. *Scientific Data*, 10, 847.
- JáJá, J. (1992). *An Introduction to Parallel Algorithms*. Addison-Wesley.
- Narouei-Khandan, H. A., et al. (2020). BLIGHTSIM: A New Potato Late Blight Model Simulating the Response of *Phytophthora infestans* to Diurnal Temperature and Humidity Fluctuations in Relation to Climate Change. *Pathogens*, 9(8), 659.
- Rocklin, M. (2015). Dask: Parallel Computation with Blocked Algorithms and Task Scheduling. *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, 126–132.
- Sentelhas, P. C., et al. (2008). Suitability of Relative Humidity as an Estimator of Leaf Wetness Duration. *Agricultural and Forest Meteorology*, 148(3), 392–400.

Apéndice A — Pseudocódigo PRAM y análisis de complejidad

Este apéndice reproduce el documento técnico de pseudocódigo en el modelo PRAM y el análisis de complejidad del pipeline. Detalla los nueve algoritmos (climatología, construcción del dataset bucket-año, fragmentación, entrenamiento de coeficientes, verificación de completitud, reparación, parchado y pronóstico), con sus costos de trabajo secuencial W , tiempo paralelo T_p y span T_∞ , así como la interpretación para cómputo de alto desempeño (HPC) y la metodología experimental completa.

2.8 A.2 Modelo PRAM

PRAM es el modelo idealizado estándar para razonar sobre algoritmos paralelos: muchos procesadores comparten una memoria, y contamos cómo se reparte el trabajo entre ellos. Usamos la variante **CREW** (Lectura Concurrente, Escritura Exclusiva):

- **Lectura concurrente:** muchos procesadores pueden leer las mismas tablas de entrada compartidas a la vez (p. ej., cada trabajador puede leer la climatología). Las lecturas no entran en conflicto.
- **Escritura exclusiva:** no hay dos procesadores que escriban en la misma ubicación de salida. Cada trabajador es dueño de un bucket / ID / día del año de salida disjunto, así que nunca hay colisiones.
- **Reducciones:** combinar m valores cuesta aproximadamente $\log m$ pasos (suma por pares ascendiendo por un árbol).
- **pardo** marca un bucle cuyas iteraciones son independientes; un planificador es libre de repartirlas entre todos los trabajadores disponibles.

2.9 A.3 Símbolos

Además de los de la Tabla 1: $T = Y \cdot D$ (total de días por ID); $R = N \cdot T$ (total de filas por variable climática); $Q \leq D$ (número de DOYs fallidos a reparar); H (horizonte de pronóstico en días). En este proyecto, D , h y F son constantes pequeñas en la práctica, pero se muestran explícitamente en las fórmulas de complejidad.

2.10 A.4 Cómo leer las fórmulas de complejidad

- **Trabajo W (costo secuencial).** El número total de operaciones básicas; lo que se pagaría en un solo procesador. Nunca disminuye al añadir hardware — es el tamaño intrínseco del cómputo.
- **Tiempo paralelo T_p .** Cuánto tarda con p trabajadores: $T_p \approx W/p_{\text{eff}} + \text{sobrecosto}$. El sobrecosto incluye reducciones ($+\log \dots$), E/S y dependencias.
- **Span T_∞ (camino crítico).** El tiempo con procesadores ilimitados; un span pequeño significa que el algoritmo es “ancho” (altamente paralelo).
- Un término $+\log(\dots)$ es el costo de una reducción paralela. **min(p, B)** aparece donde el trabajo se corta en B tareas de bucket: si $p > B$, los trabajadores sobrantes quedan ociosos ($p_{\text{eff}} \leq B$).

2.11 A.6 Algoritmo 2: Construir Climatología

La climatología es la media histórica para cada (ID, doy).

Algorithm BUILD-CLIMATOLOGY

Input: TD rows (ID, date, TD); TMIN rows (ID, date, TMIN)

Output: climatology rows (ID, doy, TD_clim, TMIN_clim)

```
1 for each climate variable V in {TD, TMIN} pardo do
2   for each monthly file m of V pardo do
3     read rows from file m
4     for each row r pardo do
5       r.doy <- day_of_year(r.date)
6       emit partial record (r.ID, r.doy, r.value)
```

```

7  for each key (ID, doy) pardo do
8      sum_V(ID, doy)  <- parallel sum of values with this key
9      count_V(ID, doy) <- parallel count of values with this key
10     clim_V(ID, doy)  <- sum_V / count_V
11  join TD_clim and TMIN_clim by (ID, doy); return climatology

```

Complejidad: $W_{\text{clim}} = O(R)$; $T_p = O(R/p + \log R)$; espacio $S_{\text{clim}} = O(ND)$ — mucho menor que la entrada cruda, porque 40 años de valores diarios colapsan en una sola curva estacional por ubicación.

2.12 A.7 Algoritmo 3: Construir el Dataset de Entrenamiento Bucket-Año

Este es el rediseño principal. La costosa fusión mensual de TD/TMIN se paga una sola vez; las filas se distribuyen luego a buckets de ID deterministas y se compactan por año.

Algorithm BUILD-BUCKET-YEAR-DATASET

Input: TD monthly files; TMIN monthly files; number of buckets B

Output: bucket-year training files: bucket b , year $y \rightarrow$ rows (ID, date, TD, TMIN, doy)

```

1  for each year y pardo do
2      for each month m in year y pardo do
3          TD_m  <- read TD rows for month m
4          TMIN_m <- read TMIN rows for month m
5          M_m   <- hash join TD_m and TMIN_m by (ID, date)
6          for each row r in M_m pardo do
7              r.doy  <- day_of_year(r.date)
8              r.bucket <- r.ID mod B
9              emit r to temporary partition (bucket = r.bucket, year = y)
10 for each bucket b in 0..B-1 pardo do
11     read all temporary rows for (bucket b, year y); sort by (ID, date)
12     write compact file for (bucket b, year y)
13 return bucket-year training dataset

```

Complejidad (joins por hash): $W_{\text{bucket}} = O(R)$; incluyendo el ordenamiento dentro de cada fragmento, $W_{\text{bucket-sort}} = O(R \log(ND/B))$; tiempo paralelo $T_p = O(R/p + \log(ND/B))$. **Memoria pico** $S_{\text{bucket-year}} = O(ND/B)$, **no** $O(R)$: un trabajador solo mantiene un fragmento bucket-año a la vez, así que aumentar B reduce la memoria que cada trabajador necesita. Esa es la palanca que evita que 300 mil IDs $\times$ 40 años tengan que caber en la RAM de una sola máquina.

2.13 A.8 Algoritmo 4: Fragmentar Climatología por Bucket

Algorithm SHARD-CLIMATOLOGY

Input: climatology rows (ID, doy, TD_clim, TMIN_clim); number of buckets B

Output: bucketed climatology files

```

1  for each climatology row c pardo do
2      c.bucket <- c.ID mod B
3      emit c to bucket c.bucket
4  for each bucket b in 0..B-1 pardo do
5      write climatology rows for bucket b
6  return bucketed climatology

```

Complejidad: $W_{\text{clim-shard}} = O(ND)$; $T_p = O(ND/p + \log B)$; memoria con streaming $S_{\text{clim-shard}} = O(ND/B)$. La fragmentación no hace aritmética alguna — solo *mueve* cada fila de climatología al bucket de su ID, de modo que el entrenamiento lea los valores normales de un ID desde el mismo bucket que sus filas de entrenamiento.

2.14 A.9 Algoritmo 4b: Fragmentar TMIN Futuro por Bucket

El pronóstico necesita la TMIN futura exógena sobre el horizonte de predicción, dispuesta por los mismos buckets que los coeficientes. Sea $R_{\text{fut}} = N * H$.

Algorithm SHARD-FUTURE-TMIN-BY-BUCKET

Input: future TMIN monthly files over the forecast horizon; number of buckets B

Output: bucketed future-TMIN files: bucket b -> rows (ID, date, TMIN)

```

1  for each month m in the horizon pardo do
2      read future TMIN rows for month m
3      for each row r pardo do
4          r.bucket <- r.ID mod B
5          emit r to temporary partition (bucket = r.bucket)
6  for each bucket b in 0..B-1 pardo do
7      write compact future-TMIN file for bucket b
8  return bucketed future TMIN

```

Complejidad: $W_{\text{fut-shard}} = O(NH)$; $T_p = O(NH/p + \log B)$; memoria $S_{\text{fut-shard}} = O(NH/B)$.

2.15 A.10 Algoritmo 5: Entrenar Coeficientes de Anomalía

Para cada ID y cada doy objetivo, ajustar una regresión ponderada sobre una vecindad circular de días del año. (Pseudocódigo extendido del Algoritmo 2 de la §2.5.)

Algorithm TRAIN-COEFFICIENTS

Input: bucket-year training dataset; bucketed climatology; local DOY half-window h; B

Output: coefficients per (ID, doy)

```

1  for each bucket b in 0..B-1 pardo do
2      train_b <- read all bucket-year training rows for bucket b
3      clim_b <- read climatology rows for bucket b
4      for each ID i in bucket b pardo do
5          X_i <- rows of train_b for ID i ; C_i <- rows of clim_b for ID i ; sort
X_i by date
6          for each row t in X_i pardo do
7              TD_anom(t) <- TD(t) - TD_clim(i, doy(t))
8              TMIN_anom(t) <- TMIN(t) - TMIN_clim(i, doy(t))
9          for each row t in X_i pardo do
10             create lag features: TD_anom_lag1(t), TD_anom_lag2(t), TMIN_anom_lag1(t)
11         for each target day d in 1..366 pardo do
12             N_d <- rows where circular_distance(doy(t), d) <= h
13             remove rows in N_d with missing target or missing features
14             if size(N_d) >= minimum_sample_count then
15                 weight(t) <- tricube(circular_distance(doy(t), d) / h)
16                 beta <- weighted_least_squares(N_d)

```

```

24             write coefficients for (ID i, doy d)
27         end for
28         write coefficient file for bucket b ; write failure log for bucket b,
if needed
30 return coefficient dataset

```

De dónde viene $O(WF^2 + F^3)$: resolver un ajuste por mínimos cuadrados ponderados implica formar la matriz de ecuaciones normales $F \times F$ ($X^T W X$) a partir de W filas de muestra ponderadas y luego resolver el sistema lineal $F \times F$ resultante (el término F^3 , el costo de una pequeña factorización matricial). Como $F = 5$ es constante, ambos términos colapsan a una constante por W , así que un ajuste cuesta $O(W) = O(Yh)$ y el entrenamiento total es $W_{\text{train}} = O(NDW) = O(NDYh)$.

Con $N \approx 3 \times 10^5$ y $D \approx 366$, son aproximadamente 1.1×10^8 ajustes independientes — el costo dominante del pipeline. El span PRAM ideal es $T_\infty = O(\log W + F^3)$: si tuviéramos procesadores ilimitados, los ajustes podrían ejecutarse todos a la vez. Con p procesadores, $T_p = O(NDYh / \min(p, B) + E/S)$. Memoria pico por tarea de bucket: $S_{\text{train-bucket}} = O(NT/B + ND/B)$ — la serie temporal de entrenamiento del bucket más su climatología.

Por qué $\min(p, B)$ y no p — y qué significa para una GPU. La unidad de planificación paralela es el bucket (nivel externo), y es lo único que $\min(p, B)$ acota. Hay un segundo nivel independiente *dentro* de cada bucket: los $\approx (N/B) D$ ajustes 5×5 son mutuamente independientes. En **CPU** cada trabajador recorre los ajustes de su bucket de uno en uno; la aceleración proviene de correr muchos buckets a la vez, por lo que se desean $B \geq 4p$. En **GPU (A100)** cada dispositivo ensambla las ecuaciones normales $X^T W X$ para todos los $\approx 10^5$ ajustes de un bucket y los resuelve en una sola llamada batched entre miles de núcleos CUDA; ese ancho de lote interno es donde reside la velocidad, y $\min(p, B)$ no lo acota, así que no se empuja B muy alto en ejecuciones con GPU. En ambos casos el trabajo total $O(NDYh)$ es idéntico; el hardware solo cambia cómo se llenan los dos niveles de paralelismo.

2.16 A.11 Algoritmo 6: Verificar Completitud

Algorithm CHECK-COMPLETENESS

Input: coefficients; expected ID count from grid

Output: incomplete DOYs

```

1 expected_count <- number of IDs in grid
2 for each coefficient row c pardo do
3     emit key c.doy with value c.ID
5 for each doy d in 1..366 pardo do
6     observed_count(d) <- distinct count of IDs emitted for d
7     if observed_count(d) < expected_count then
8         mark d as incomplete
11 return incomplete DOYs

```

Con $C = ND$: $W_{\text{check}} = O(C)$; $T_p = O(C/p + \log C)$; espacio $S_{\text{check}} = O(C)$. Un ajuste (ID, doy) puede omitirse si su vecindad tuvo demasiadas pocas muestras válidas (línea 19 del entrenamiento); este paso es una auditoría de cordura cuya salida es diminuta — una breve lista de a lo sumo D días a rehacer.

2.17 A.12 Algoritmo 7: Reparar DOYs Fallidos

Solo se reentrenan los DOYs incompletos. Si Q es pequeño, esto es mucho más barato que reentrenar los 366 DOYs.

Algorithm RERUN-FAILED-DOYS

Input: bucket-year training dataset; bucketed climatology; incomplete DOYs Q ; half-window h

Output: patch coefficient dataset

```

1  for each bucket  $b$  in  $0..B-1$  pardo do
2      train_b <- read all bucket-year training rows for bucket  $b$ 
3      clim_b  <- read climatology rows for bucket  $b$ 
4      for each ID  $i$  in bucket  $b$  pardo do
5          recompute anomalies and lags for ID  $i$ 
6          for each failed day  $d$  in  $Q$  pardo do
7              fit weighted local regression for (ID  $i$ , doy  $d$ )
8              write patch coefficients for (ID  $i$ , doy  $d$ )
12 return patch coefficient dataset

```

Complejidad: $W_{\text{repair}} = O(NQYh)$; $T_p = O(NQYh/p + \log(Yh))$. La reparación es estructuralmente idéntica al entrenamiento pero restringida a los Q días marcados; cuando solo falló un puñado de días ($Q \ll D$), esto es una fracción diminuta de un reentrenamiento completo.

2.18 A.13 Algoritmo 8: Parchar Coeficientes

Algorithm PATCH-COEFFICIENTS

Input: base coefficient dataset; patch coefficient dataset; incomplete DOYs Q

Output: corrected coefficient dataset

```

1  for each bucket  $b$  in  $0..B-1$  pardo do
2      base_b      <- read coefficient rows for bucket  $b$ 
3      patch_b     <- read patch rows for bucket  $b$ 
4      base_keep   <- rows in base_b whose doy is not in  $Q$ 
5      corrected_b <- union(base_keep, patch_b)
6      remove duplicate (ID, doy), keeping the patch row
7      sort corrected_b by (ID, doy); write corrected coefficients for bucket  $b$ 
10 return corrected coefficient dataset

```

Complejidad: $W_{\text{patch}} = O(ND + NQ)$; $T_p = O((ND + NQ)/p)$; memoria $S_{\text{patch-bucket}} = O(ND/B + NQ/B)$. Es pura contabilidad —una lectura, un filtro, una unión, un ordenamiento y una escritura por bucket—, sin nuevo ajuste de modelo.

2.19 A.14 Algoritmo 9: Pronóstico Opcional (bucketizado)

El pronóstico es paralelo entre buckets, y entre IDs dentro de un bucket, pero **recursivo a través del tiempo para cada ID**: el día t depende de la anomalía de TD predicha previamente, así que el bucle del horizonte no puede paralelizarse dentro de un mismo ID.

Algorithm FORECAST-TD

Input: bucketed coefficients, climatology, history (to seed lags), future TMIN; horizon H ; B

Output: predicted TD for each (ID, date)

```

1  for each bucket  $b$  in  $0..B-1$  pardo do
2      coeff_b, clim_b, hist_b, ftmin_b <- read bucket  $b$  shards

```

```

6   for each ID i in bucket b pardo do
7     initialize lagged TD/TMIN anomalies from hist_b(i)
8     for forecast day t from 1 to H do           // secuencial en t
9       d <- day_of_year(date_t) ; beta <- coeff_b(i, d)
11      x <- current TMIN anomaly and lagged anomaly features
12      TD_anom_hat <- beta dot x
13      TD_hat <- clim_b(i, d) + TD_anom_hat
14      update lagged anomalies with TD_anom_hat and TMIN_anom(t)
15      write prediction (ID i, date_t, TD_hat)
18  write prediction file for bucket b
20  return predictions

```

Complejidad: $W_{\text{forecast}} = O(NH)$; tiempo práctico $T_p = O(NH / \min(p, B) + H)$. El término $+H$ permanece porque la recursión de pronóstico es inevitable por **ID**; aún paralelizamos entre los millones de ubicaciones (y entre buckets), pero el avance temporal por **ID** sigue siendo secuencial.

2.20 A.15 Diseño antiguo vs diseño bucket-año

El diseño original al estilo Colab entrenaba una tarea por **ID**. Cada tarea descubría, abría y filtraba los mismos archivos parquet mensuales. Sea $S = 12Y$ el número de archivos fuente mensuales por variable. El sobrecosto de E/S por tarea del diseño antiguo es $O(NS)$; para $N = 300\,000$ y $Y = 40$ esto significa aproximadamente $300\,000 \times 480 = 144\,000\,000$ intentos de acceso a archivos. El diseño bucket-año escanea los archivos fuente **una vez** durante el preprocesamiento ($O(S)$), y el trabajo de entrenamiento del modelo es $O(NDYh)$. El rediseño no elimina el trabajo estadístico, pero sí elimina el escaneo repetido que hacía lenta y frágil la implementación anterior.

2.21 A.17 Complejidad global

Usando $R = NYD$, el trabajo secuencial total para el preprocesamiento más el entrenamiento completo (sin reparación ni pronóstico) es

$$W_{\text{total}} = \left(OR \log \frac{ND}{B} + NDYh \right) = O(NYD + NDYh) = O(NDYh),$$

porque h es el principal multiplicador de la regresión local. La expresión de extremo a extremo más completa (con reparación de Q DOYs y pronóstico de horizonte H) es

$$W_{\text{end-to-end}} = \left(QNYD \log \frac{ND}{B} + NYh(D + Q) + NH \right),$$

y el tiempo paralelo práctico es

$$T_p = \left(O \frac{NYD \log(ND/B)}{p} + \frac{NDYh}{\min(p, B)} + \frac{NQYh}{\min(p, B)} + \frac{NH}{\min(p, B)} + H + \log(NYD) \right).$$

Las fases de preprocesamiento se dividen por p (limitadas por el sistema de archivos), mientras que las tres etapas de cómputo se dividen por $\min(p, B)$. El cuello de botella práctico tras el rediseño es $O(NDYh)$.

2.22 A.18 Interpretación HPC

El trabajo total no depende de p porque mide el número total de operaciones requeridas por el algoritmo: los mismos joins, cálculos de anomalías, regresiones locales, reparaciones y pronósticos deben computarse tanto en un procesador como en un clúster HPC. La medida relevante para el tiempo

de ejecución es $T_p = O(W/p_{\text{eff}} + E/S + \text{planificación} + \text{dependencias})$, donde p_{eff} es el número efectivo de procesadores que el algoritmo puede usar. Para el diseño bucket-año, $p_{\text{eff}} \leq B$: si $B = 1024$, el entrenamiento expone hasta ~ 1024 tareas de bucket paralelas. Si el HPC ofrece muchas más ranuras, los procesadores extra no reducen el tiempo a menos que se aumente B o se añada otro nivel de paralelismo dentro de cada bucket. Una regla práctica para un despliegue HPC es $B \geq 4p$.

Tabla 3: Puntos de partida sugeridos para el número de buckets.

Ranuras de trabajador planificadas	Número de buckets sugerido
64	512 o 1024
128	1024 o 2048
256	2048 o 4096
512	4096 o 8192

La aceleración esperada es fuerte mientras p_{eff} aumenta, pero acaba saturándose debido al ancho de banda de E/S, el sobrecosto del planificador, el número de buckets y la dependencia recursiva del pronóstico sobre el horizonte H .

2.23 A.19 El hardware es simplemente más procesadores

La complejidad de trabajo total $W = O(NDYh)$ es la misma tanto si el pipeline corre en núcleos de CPU como en una GPU. El hardware cambia solo dos cosas: el número de procesadores paralelos p que expone, y el factor constante c de un solo ajuste local. El tiempo de ejecución paralelo es siempre $T_p = c \cdot (W/p_{\text{eff}}) + \text{sobrecosto}$.

Los mínimos cuadrados ponderados por (ID, doy) son vergonzosamente paralelos: hay unos $N \cdot D \approx 1.1 \times 10^8$ ajustes independientes, cada uno una pequeña regresión ponderada de $F = 5$ características sobre $W = O(Yh)$ filas de vecindad. Esto se mapea igual de bien a:

- **CPU**: muchos procesos trabajadores de bucket de un solo hilo (uno por núcleo), donde la estructura de buckets acota $p_{\text{eff}} \leq B$; o
- **GPU**: un dispositivo que evalúa miles de estos ajustes diminutos concurrentemente una vez agrupados en lotes, ensamblando las ecuaciones normales $F \times F$ $(X^\top W X)\beta = X^\top W y$ para muchos pares (ID, doy) y resolviéndolos en una sola llamada batched.

Cuál de CPU o GPU produce el menor $c \cdot T_p$ para esta carga es una pregunta empírica, respondida por los experimentos de escalamiento, no por el modelo de complejidad. Las aceleraciones restantes, independientes del hardware, siguen viniendo de: evitar escaneos repetidos de parquet (el rediseño bucket-año), usar archivos bucket-año, mantener acotada la memoria del trabajador, paralelizar por bucket, y reparar solo los DOYs fallidos en lugar de repetir todo el entrenamiento.

2.24 A.20 Metodología experimental

Esta sección define cómo se evalúa la implementación en el HPC universitario. El objetivo es medir, como función del número de procesadores p , tres cantidades estándar de rendimiento paralelo —tiempo de ejecución $T(p)$, aceleración $S(p) = T(1)/T(p)$ y eficiencia $E(p) = S(p)/p$ — tanto en hardware de CPU como de GPU y en ambas versiones del dataset.

Definición de un “procesador” p . Para comparar CPU y GPU en un mismo eje, p cuenta ranuras de trabajador independientes: en CPU (SLURM) un trabajador es un proceso Dask de un solo hilo (un núcleo, BLAS fijado a 1), escalado con `dask_jobqueue.SLURMCluster`; en GPU un trabajador es una GPU A100 (`dask-cuda`), y p es el número de A100 de la asignación.

Controles. La línea base $T(1)$ usa un solo trabajador con `threads_per_worker = 1` y los pools de BLAS fijados a 1; el número de buckets se mantiene en $B \geq 4 p_{\max}$; cada punto (p, n) se ejecuta k veces (p. ej. $k = 3$) reportando la mediana; el almacenamiento temporal va a disco local rápido del nodo; se distingue caché caliente vs fría; y solo se cronometra la fase bajo prueba.

Escalamiento fuerte. Problema fijo, $p \in \{1, 2, 4, 8, 16, 32, \dots\}$ (CPU) y $p \in \{1, 2, 4, 8\}$ A100 (GPU); curva casi lineal mientras $p \leq B$, luego se aplanan (prueba directa de $p_{\text{eff}} \leq B$).

Escalamiento débil. Trabajo por procesador constante, $n(p) = n_0 \cdot p$; eficiencia $E_{\text{weak}}(p) = T(1, n_0)/T(p, n_0 p)$, idealmente 1.

Datasets y reportes. El protocolo completo se ejecuta sobre PISCOT v1.1 y v1.2, suministradas al mismo pipeline mediante parámetros de ruta. Se compara el **rendimiento** (curvas T, S, E) y la **ciencia** (distribuciones de los coeficientes ajustados y de la habilidad predictiva), para evaluar cómo la versión del dataset afecta al modelo estimado con independencia del tiempo de ejecución.